

PantryPilot Configuration Management Report

Prepared by	Zixuan Liang
Course	CISC 594: Software Testing Principles and Techniques
Instructor	Dr. Khalid Lateef
Project	PantryPilot - Smart Meal and Grocery Planner
Submission	HU14 Final Project: Configuration Management Report
Date	August 4, 2026

Executive Summary

This report documents how configuration management was applied to PantryPilot, the semester project for CISC 594. The project source code was maintained in a Git repository with an approved baseline on main, a separate feature branch for new development, a test-before-merge workflow, and annotated release tags for completed versions. A separate source-code ZIP is submitted with this report so the repository contents, source files, tests, CI configuration, branches, and tags can be inspected.

The main configuration-management result is that PantryPilot has two traceable baselines: v1.0.0 for the initial Plan & Adapt CM baseline and v1.1.0 for the controlled Safety & Grocery Controls update. The v1.1.0 update was developed on feature/safety-grocery-controls, tested locally, merged back to main, and tagged after verification.

Project Context and CM Objectives

PantryPilot is a smart meal and grocery planner that maintains dietary constraints, recipe data, pantry inventory, weekly plans, grocery lists, and AI-assisted substitutions. Because the project handles safety-sensitive constraints such as allergens and expiry dates, configuration management is needed to keep source code, requirements, tests, configuration files, and release records synchronized.

The CM objective was to make every change traceable from a reason for change to branch work, test evidence, merge, release tag, and rollback point. This matches the assignment requirement to use version control, develop outside the main branch, test changes before merging, and tag completed software versions.

Repository and Tool Use

Table 1

Version-control evidence

CM item	Evidence	Purpose
Version-control tool	Git repository in PantryPilot_CM_Generic_Files; source ZIP includes repository history.	Provides auditable history, branch control, tags, and rollback.
Main branch	main	Approved working baseline after tested changes are merged.
Development branch	feature/safety-grocery-controls	Isolated new development from the baseline until verification.
Release tags	v1.0.0 and v1.1.0	Identify tested software versions and rollback points.
CI workflow	.github/workflows/ci.yml	Defines automated pytest gate for pushes and pull requests to main.
Instructor access	Separate source-code ZIP submitted with .git history; repository can be pushed to GitHub and shared read-only.	Allows review of source, tests, branches, tags, and release records.

Configuration Items

The project treats source files, tests, runtime configuration, CM documentation, CI workflow, release notes, and version files as controlled configuration items. These items are tracked together because a release is only meaningful if the code, tests, configuration, and release description agree.

Table 2
Controlled configuration items

Configuration item	Location	Control method	Release relevance
Source code	src/pantrypilot_app.py	Branch, commit, review, test	Implements release behavior.
Tests	tests/test_smoke.py	Required before merge	Demonstrates baseline quality gates.
Runtime configuration	config/app_config.json	Version controlled	States baseline version and enabled features.
Release identity	VERSION, CHANGELOG.md	Updated with each release	Keeps version status accounting current.
Release notes	releases/	One file per tagged release	Documents scope, verification, and rollback.
CI workflow	.github/workflows/ci.yml	Protected project file	Defines repeatable verification command.
CM documentation	docs/	Reviewed before baseline	Explains branching and change-control policy.

Formal Change-Control Process

The formal change-control process is intentionally lightweight but complete enough for the semester project. A change is not merged just because it compiles; it must have a stated purpose, a branch, test evidence, updated configuration records, and a rollback path.

Table 3

Change-control workflow

Step	Activity	PantryPilot evidence	Control value
1	Identify change request	Need to add safety filtering, grocery aggregation, and expiry boundary behavior.	Prevents unmanaged scope drift.
2	Create branch from main	feature/safety-grocery-controls	Keeps new development separate from baseline.
3	Implement and update configuration items	Source, tests, VERSION, config, changelog, and release notes changed together.	Keeps code and records synchronized.
4	Test on branch	6 test functions passed through offline smoke runner; CI workflow is configured for pytest.	Verifies before merge.
5	Merge to main	Merge commit 834e64e	Moves only tested change into approved baseline.
6	Tag release	Annotated tag v1.1.0	Creates a recoverable version reference.
7	Record status accounting	CHANGELOG.md and releases/release_notes_v1.1.0.md	Documents what changed and how to roll back.

Branching, Merge, and Tagging Evidence

The repository history demonstrates the required branch workflow. The initial baseline was committed on main and tagged v1.0.0. New code was then developed on feature/safety-grocery-controls, committed as a branch change, merged back to main, and tagged v1.1.0 after verification.

Table 4

Repository history

Commit / tag	Branch	Description	CM meaning
842ae5e; tag v1.0.0	main	Establish PantryPilot CM baseline	Initial approved baseline.
72a997e	feature/safety-grocery-controls	Add safety and grocery control tests	Controlled branch implementation.
834e64e; tag v1.1.0	main	Merge safety and grocery controls	Merged tested release baseline.

Commit graph evidence: main at 834e64e with tag v1.1.0; feature/safety-grocery-controls at 72a997e; original baseline 842ae5e with tag v1.0.0.

Testing and Release Verification

Testing is part of the CM gate. The GitHub Actions workflow installs requirements.txt and runs pytest for pushes and pull requests to main. In the local environment used to prepare the submission, pytest was not installed, so the same test functions were executed through an offline smoke runner. The result was: 6 tests passed.

The v1.1.0 tests cover release description, quality gates, allergen alias detection, safe recipe approval, grocery aggregation, and pantry-expiry boundary behavior. These tests are tied to the project risks from the risk-management report: unsafe AI output, grocery-list correctness, and date-boundary defects.

Baseline, Release, and Rollback Control

Table 5
Baseline and release records

Version	Release name	Tag	Verification	Rollback target
1.0.0	Plan & Adapt	v1.0.0	Initial source/config/test baseline	N/A
1.1.0	Safety & Grocery Controls	v1.1.0	6 tests passed locally; CI workflow configured	v1.0.0

Rollback is simple because tags identify recoverable baselines. If v1.1.0 fails release acceptance, the project can return to tag v1.0.0 while the feature branch is corrected and retested.

Repository Access and Source-Code Submission

The separate source-code ZIP submitted with this report contains the PantryPilot source package and its Git metadata. The instructor can inspect tracked files, branch history, annotated tags, tests, CI workflow, release notes, and the CM documentation. If the class workflow requires a live GitHub repository instead of a ZIP-only review, the same repository should be pushed to GitHub and Dr. Lateef should be added as a read-only collaborator.

No secrets are stored in the submitted source. Environment-specific values belong in `config/environment.example` or in local environment variables, not in committed files. The `.gitignore` excludes `pycache` files, `pytest` cache, virtual environments, logs, and `.env` files.

Conclusion

PantryPilot's CM process satisfies the assignment by using version control, isolating new development on a feature branch, testing before merge, merging the verified change to main, tagging completed versions, and documenting change-control records. The process also supports software quality because the release tags, tests, release notes, and rollback path make the project auditable and recoverable.

References

- Liang, Z. (2026a). PantryPilot configuration management generic files [Course assignment]. Harrisburg University of Science and Technology.
- Liang, Z. (2026b). PantryPilot risk management report [Course assignment]. Harrisburg University of Science and Technology.
- Liang, Z. (2026c). PantryPilot final project presentation [Course presentation]. Harrisburg University of Science and Technology.